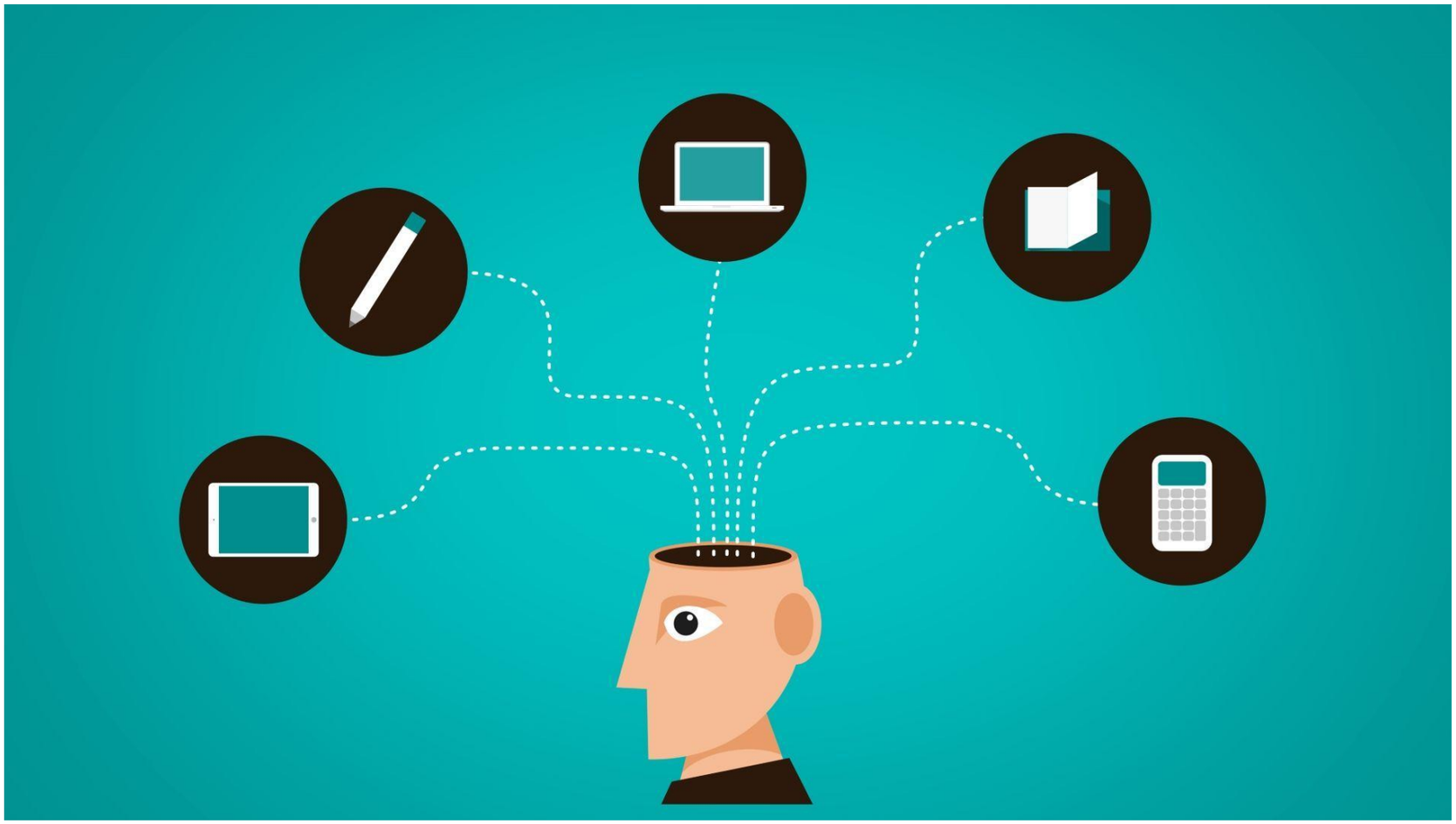


Теория графов

Связность

Генерация графов

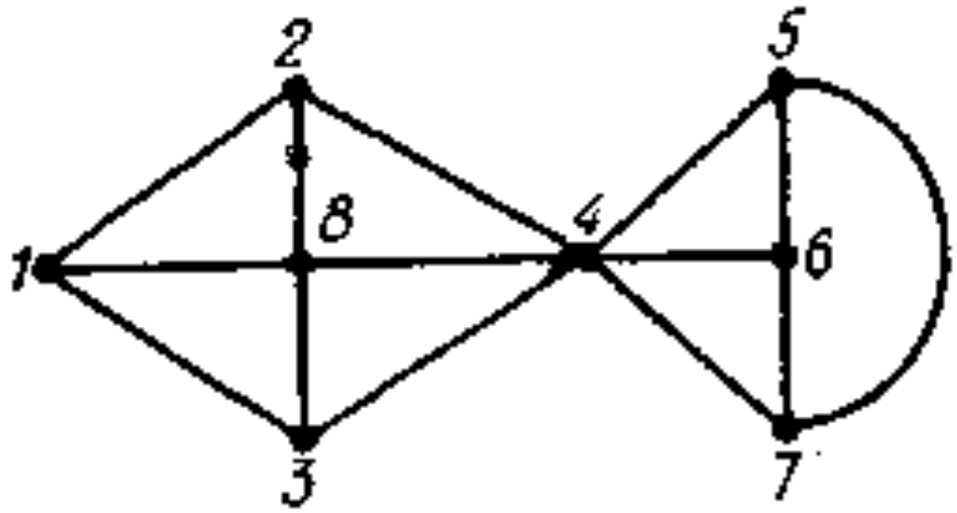
Начнем с примера

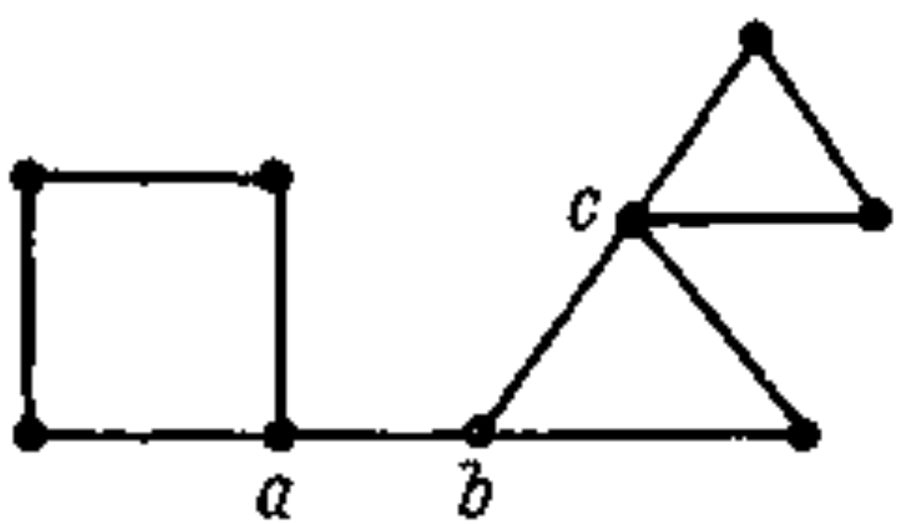
Рассмотрим математическую модель, возникающую, в частности, при проектировании и анализе сетей ЭВМ.

- Имеется сеть, состоящая из центров хранения и переработки информации. Некоторые пары центров соединены каналами.
- Обмен информацией между любыми двумя центрами осуществляется либо непосредственно по соединяющему их каналу, если он есть, либо через другие каналы и центры.
- Сеть считается исправной, если каждая пара центров в состоянии обмениваться информацией.
- Исправной сети будет соответствовать связный граф.
- ***Надежность (живучесть)*** сети - способность сети функционировать при выходе из строя одного или нескольких центров или (и) каналов. Менее надежной следует считать ту сеть, исправность которой нарушается при повреждении меньшего количества элементов.

НАСКОЛЬКО СЕТЬ НАДЕЖНА?

Пример





Связность

- *Т Е О Р Е М А. Граф связен тогда и только тогда, когда его нельзя представить в виде объединения двух графов.*

Доказательство

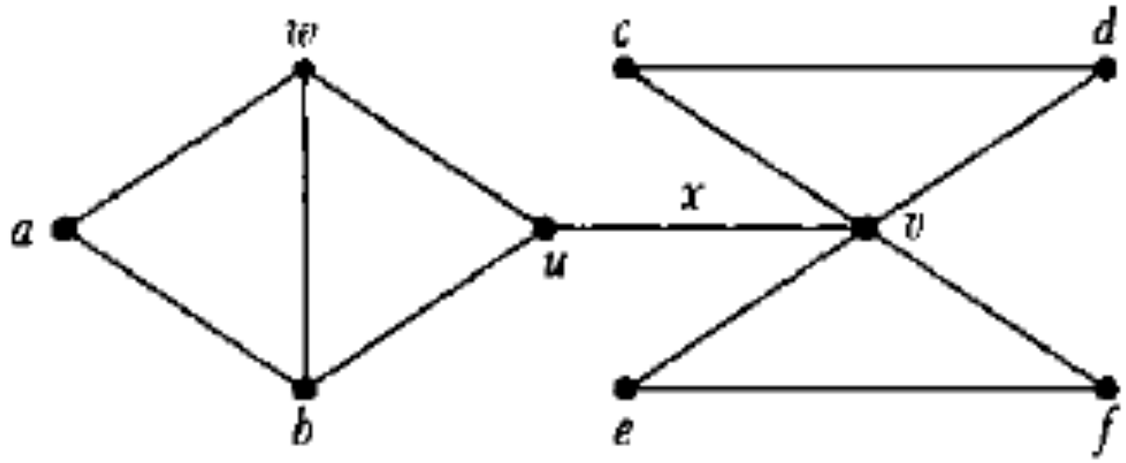
[$\Longrightarrow$] От противного. Пусть $k(G) = 1$ и граф G состоит из двух компонент, то есть $G = G_1(V_1, E_1) \cup G_2(V_2, E_2)$, где $V_1 \cap V_2 = \varnothing$, $E_1 \cap E_2 = \varnothing$, $V_1 \neq \varnothing$, $V_2 \neq \varnothing$. Возьмём $v_1 \in V_1$, $v_2 \in V_2$. Тогда $\exists \langle v_1, v_2 \rangle$ $(v_1 \in V_1 \ \& \ v_2 \in V_2)$. В этой цепи $\exists e = (a, b)$ $(a \in V_1 \ \& \ b \in V_2)$. Но тогда $e \notin E_1$, $e \notin E_2$, следовательно, $e \notin E$.

[$\Longleftarrow$] От противного. Пусть $k(G) > 1$. Тогда существуют две вершины u и v, не соединённые цепью. Следовательно, вершины u и v принадлежат разным компонентам связности. Положим G_1 — компонента связности для u, а G_2 — все остальное. Имеем $G = G_1 \cup G_2$. $\square$

- Несвязный граф всегда можно представить как объединение связных компонент. Эти компоненты можно рассматривать независимо. Поэтому во многих случаях можно без ограничения общности предполагать, что рассматриваемый граф связен.

Точки сочленения, мосты и блоки

- Вершина графа называется *точкой сочленения*, или *разделяющей вершиной*, если её удаление увеличивает число компонент связности.
  / вершины u, v — точки сочленения, и других точек сочленения нет /
- *Мостом* называется ребро, удаление которого увеличивает число компонент связности.
  / ребро x — мост, и других мостов нет /
- *Блоком* называется связный граф, не имеющий точек сочленения.
  / правильные подграфы{a, b, w}, {b,u,w}, {a,b,u,w}, {c, d, v}, {e,f,v} — блоки, и других блоков нет /



Точки сочленения, мосты и блоки

ЛЕММА *В любом нетривиальном графе есть по крайней мере две вершины, которые не являются точками сочленения.*

ТЕОРЕМА 1 *Пусть $G(V, E)$ — связный граф и $v \in V$. Тогда следующие утверждения эквивалентны:*

1. *v — точка сочленения.*
2. $\exists\, u, w \in V \ (u \neq w \ \& \ \forall \ \langle u, w\rangle_G \ (v \in \langle u, w\rangle))$.
3. $\exists\, U, W \ (U \cap W = \varnothing \ \& \ U \cup W = V - v \ \& \ \forall\, u \in U, w \in W \ (\forall \langle u, w\rangle_G \ (v \in \langle u, w\rangle_G)))$.

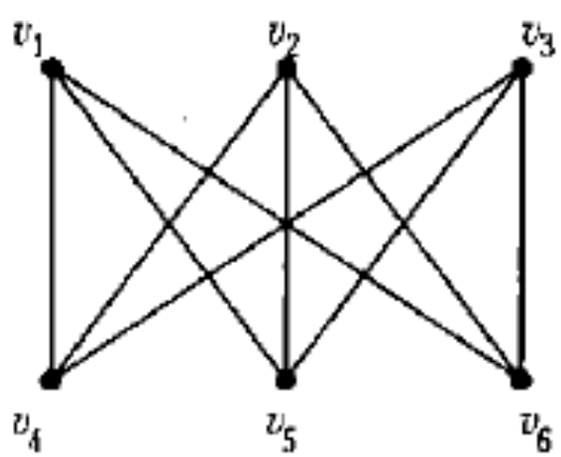
СЛЕДСТВИЕ *Если вершина инцидентна мосту и не является висячей, то она является точкой сочленения.*

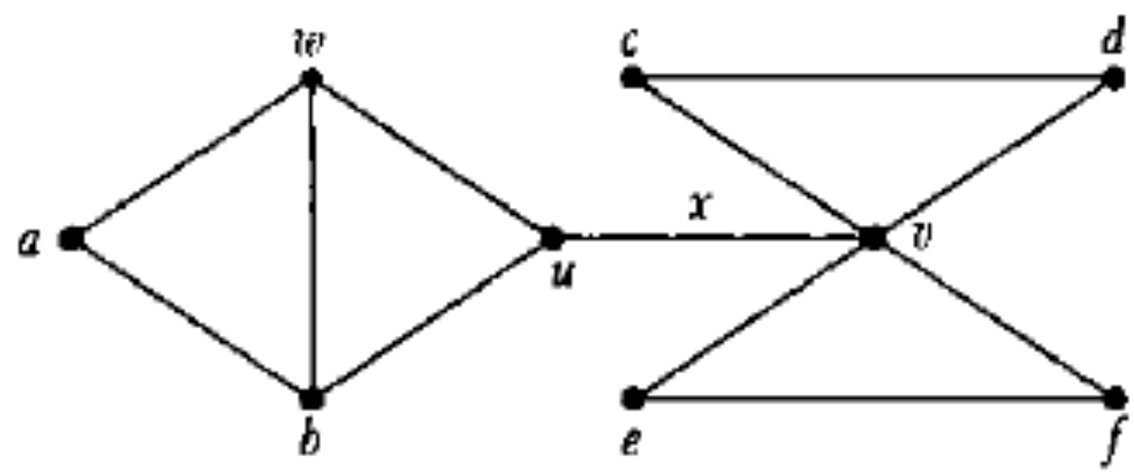
ТЕОРЕМА 2 *Пусть $G(V, E)$ — связный граф и $x \in E$. Тогда следующие утверждения эквивалентны:*

1. *x — мост.*
2. *x не принадлежит ни одному простому циклу.*
3. $\exists\, u, w \in V \ (\forall \langle u, w\rangle_G \ (x \in \langle u, w\rangle_G))$.
4. $\exists\, U, W \ (U \cap W = \varnothing \ \& \ U \cup W = V \ \& \ \forall\, u \in U \ (\forall\, w \in W \ (\forall \langle u, w\rangle_G \ (x \in \langle u, w\rangle)_G)))$.

Меры связности

- При взгляде на диаграммы связных графов возникает естественное впечатление, что связный граф может быть «более» или «менее» связен. Введем некоторые количественные меры связности.
- *Вершинной связностью* графа ***G*** (обозначается χ***(G))*** называется наименьшее число вершин, удаление которых приводит к несвязному или тривиальному графу.





Вершинная связность

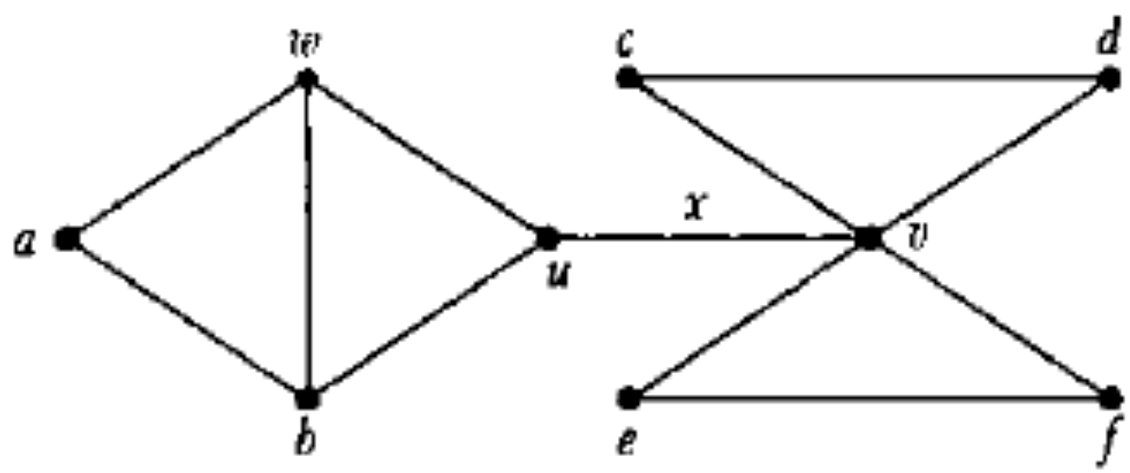
Пример

$\varkappa(G) = 0$, если G несвязен;

$\varkappa(G) = 1$, если G имеет точку сочленения;

$\varkappa(K_p) = p - 1$, если K_p — полный граф.

Граф G называется *n-связным*, если $\varkappa(G) = n$.

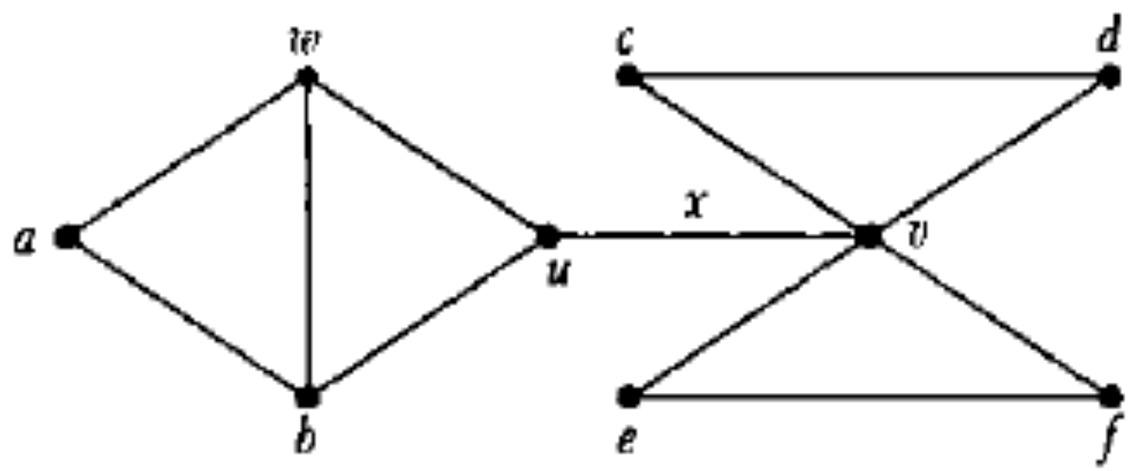


Реберная связность

- *Рёберной связностью* графа *G* (обозначается λ (G)) называется наименьшее число рёбер, удаление которых приводит к несвязному или тривиальному графу.

Пример

$\lambda(G) = 0$, если G несвязен;
$\lambda(G) = 1$, если G имеет мост;
$\lambda(K_p) = p - 1$, если K_p — полный граф.



Оценка числа рёбер

ТЕОРЕМА $\varkappa(G) \leqslant \lambda(G) \leqslant \delta(G)$.

$$p - k \leqslant q \leqslant \frac{(p-k)(p-k+1)}{2}.$$

где p — число вершин, q — число рёбер, к — число компонент связности графа.

СЛЕДСТВИЕ 1 *Если* $q > (p-1)(p-2)/2$, *то граф связен.*

СЛЕДСТВИЕ 2 *В связном графе* $p - 1 \leqslant q \leqslant p(p-1)/2$.

Алгоритм нахождения блоков и точек сочленения

```
procedure BiComp(v);
begin
  num[v] := i; L[v] := i; i := i + 1;
  for u ∈ list[v]  do
    if num[u] = 0 then
      begin
        SE ⇐ vu; father[u] := v; BiComp(u)
        L[v] := min(L[v], L[u]);
        if L[u] ≥ num[v] then
          {получить новый блок; для этого из
           стека SE надо вытолкнуть все ребра
      end
    else if num[u] < num[v] and u ≠ father[v] then
      begin
        SE ⇐ vu; L[v] := min(L[v], num[u]);
      end
end;
begin
  i := 1; SE := nil; father[v0] := ∅;
  for v ∈ V  do  num[v] := 0;
  BiComp(v0)
end.
```

Двусвязные и трехсвязные графы

Случаям, когда k = 2 или k = 3, в теории графов отведена особая роль.

- Во-первых, 2- и 3-связные графы фигурируют во многих теоретических и прикладных вопросах, в частности, ряд задач достаточно уметь решать для 2-связных компонент.
- Во-вторых, при k = 3 и, особенно, при k = 2 удается дать в некоторой степени обозримое описание соответствующих графов.

При решении многих задач на графах достаточно уметь решать эти «задачи для каждой 2-связной компоненты графа. Поэтому представляет интерес взаимное расположение 2-компонент в графе.

Почти все графы двусвязны.

Почти все графы не содержат мостов.

Свойства двусвязных графов

1. степени вершин 2-связного графа больше единицы;
2. если графы G_1 и G_2 2-связны и имеют не менее двух общих вершин, то граф $G_1 \cup G_2$ также 2-связен;
3. если граф G 2-связен и P — простая цепь, соединяющая две его вершины, то граф $G \cup P$ также 2-связен;
4. если вершина v не является точкой сочленения связного графа, то любые две его вершины соединены цепью, не содержащей v, в частности, в 2-связном графе для любых трех несовпадающих вершин a, b, v имеется (a, b)-цепь, не проходящая через v.

Теорема

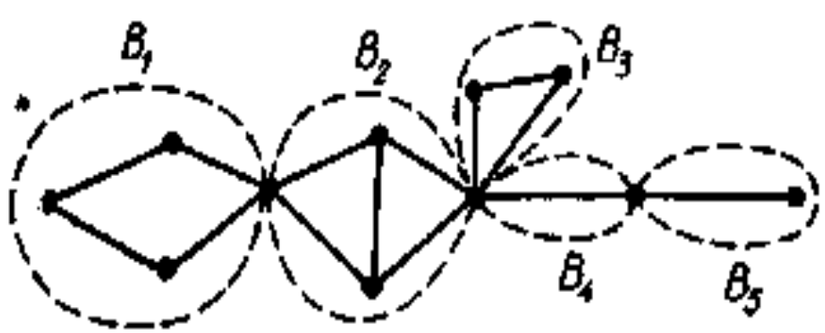
Пусть G—связный граф и |G| > 2. Тогда следующие утверждения эквивалентны'.

1. граф 2-связен;
2. любые две вершины графа принадлежат простому циклу;
3. любая вершина и любое ребро принадлежат простому циклу;
4. любые два ребра принадлежат простому циклу;
5. для любых двух вершин a и b и любого ребра e существует простая (a, b)-цепъ, содержащая e;
6. для любых трех вершин a, b, c существует простая (a, b)-цепь, проходящая через c.

Блоковые множества

Каждый блок графа либо 2-связен, либо совпадает с K_2 или с K_1 (граф K_1 — блок тогда и только тогда, когда он является связной компонентой). Связный граф без точек сочленения также называют блоком. Множество вершин блока - *блоковое множество*.

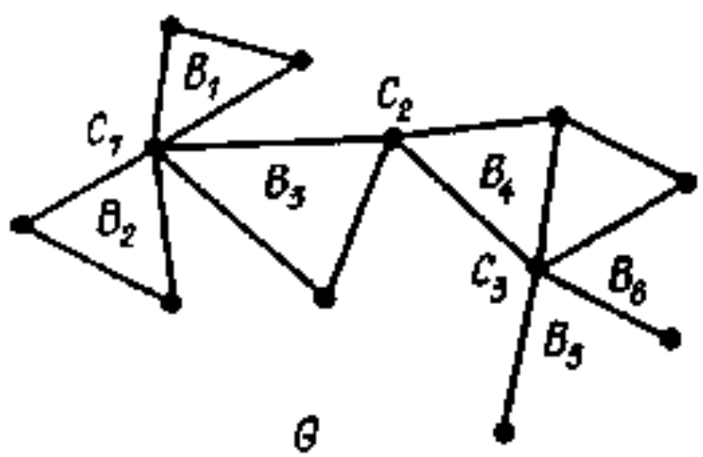
- Любые два блока графа имеет не более одной общей вершины. В частности, всякое ребро графа входит только в один его блок.
- Если блок графа содержит вершины a и b, то он содержит и всякую простую (a, b)-цепь этого графа.
- Если вершина v входит более чем в один блок графа G, то v — точка сочленения этого графа.

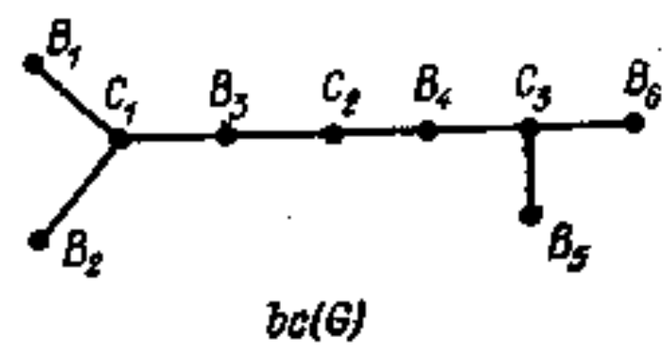


Среди блоков B_1, B_2 и B_3 — 2-связные графы, а каждый из двух оставшихся является ребром.

Блоковая структура графа

- Система блоковых множеств графа является покрытием множества его вершин.
- Каждая пара блоковых множеств либо не пересекается, либо имеет единственную общую вершину, и эта вершина является точкой сочленения графа.
- Структура графа *«с точностью до блоков».*
- *bc(G) – граф, дерево.* Блоки графа G, соответствующие концевым вершинам его bc-дерева, называются концевыми блоками.



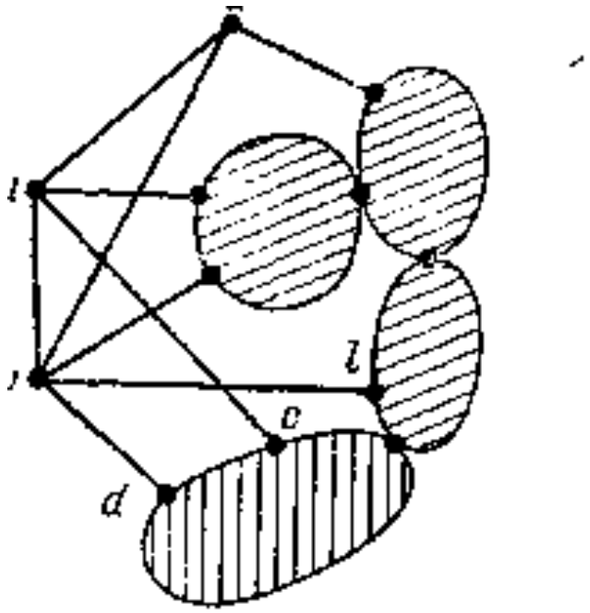


Свойства трехсвязных графов

При любом выборе ребра $uv \in EG$ граф G_{uv} обладает следующими свойствами:

- если a — висячая вершина графа G_{uv}, то $av \in EG$, $au \in EG$;
- всякий висячий блок графа G_{uv}, не являющийся ребром, содержит такую пару вершин c и d, отличных от точек сочленения графа G_{uv}, что $uc \in EG$, $vd \in EG$;
- всякий блок графа G_{uv}, имеющий ровно две точки сочленения и отличный от ребра, содержит такую вершину l, не являющуюся точкой сочленения графа G_{uv}, что $u_l \in EG$ или $v_l \in$. EG.

Во всяком 3-связном графе G существует такое ребро uv, что граф G_{uv}, не имеет точек сочленения.
Всякий 3-связный граф с числом вершин $n \geq 5$ содержит ребро, стягивание которого приводит к 3-связному графу.



Цепи и разрезы

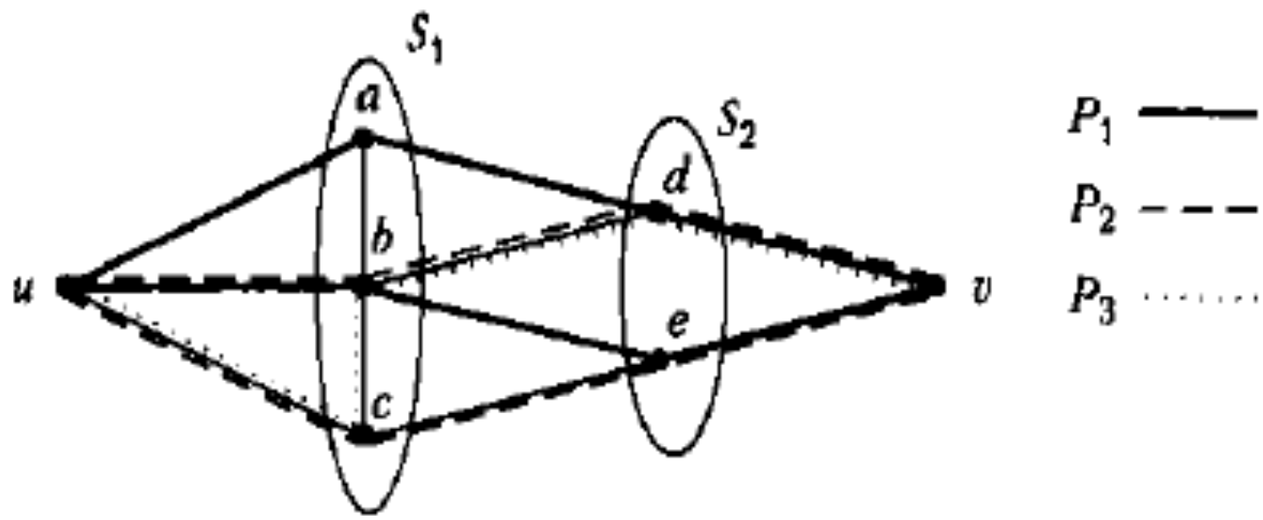
- Интуитивно очевидно, что граф тем более связен (при использовании различных мер связности), чем больше существует различных цепей, соединяющих одну вершину с другой, и тем менее связен, чем меньше нужно удалить промежуточных вершин, чтобы отделить одну вершину от другой.
- Пусть *G(V, E)* — связный граф, ***u*** и ***v*** — две его несмежные вершины. Две цепи **<*u, v*>** называются *вершинно-непересекающимися*, если у них пет общих вершин, отличных от ***u*** *и* ***v***. Две цепи **<*u, v*>** называются *рёберно-непересекающимися*, если у них нет общих рёбер. Если две цепи вершинно не пересекаются, то они и рёберно не пересекаются.
- Обозначим через ***P(u, v)*** множество вершинно-непересекающихся цепей **<*u, v*>** .
- Множество *S* вершин (и/или рёбер) графа *G разделяет* две вершины *unv*, если ***u*** и ***v*** принадлежат разным компонентам связности графа ***G - S***.
- *Разделяющее множество рёбер* называется *разрезом. Разделяющее множество вершин* для вершин ***u*** и ***v*** обозначим ***S(u,v)***.
- Для заданных вершин ***u*** *и* ***v*** множества *P(u, v)* и *S(u, v)* можно выбирать различным образом.

$$\bigcap_{\langle u,v\rangle \in P(u,v)} \langle u, v\rangle = \{u, v\},$$

$$G - S = G_1 \cup G_2 \ \&\ v \in G_1 \ \&\ u \in G_2.$$

Пример

Если u и v принадлежат разным компонентам связности графа G, то $|P(u,v)| = 0$ и $|S(u,v)| = 0$. Поэтому без ограничения общности можно считать, что G — связный граф.

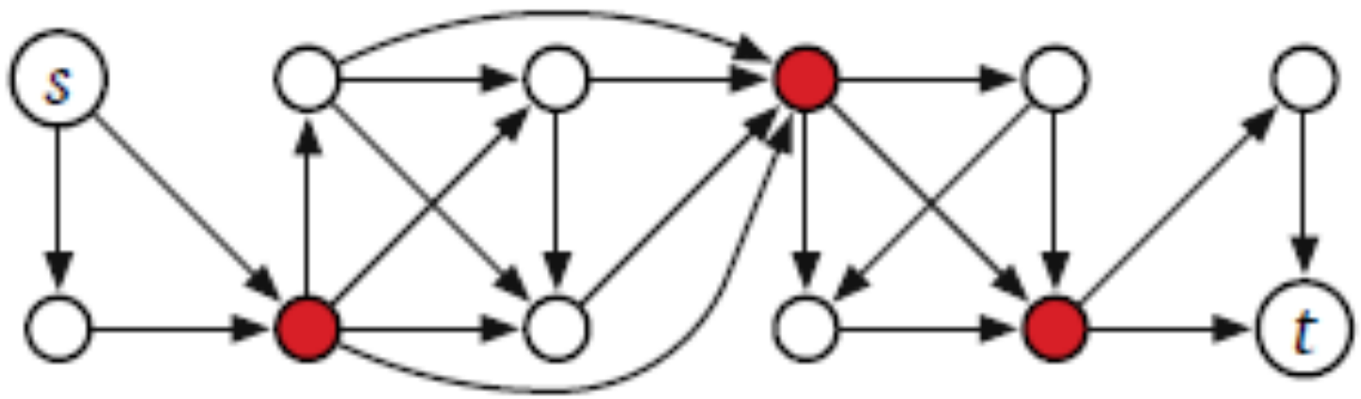


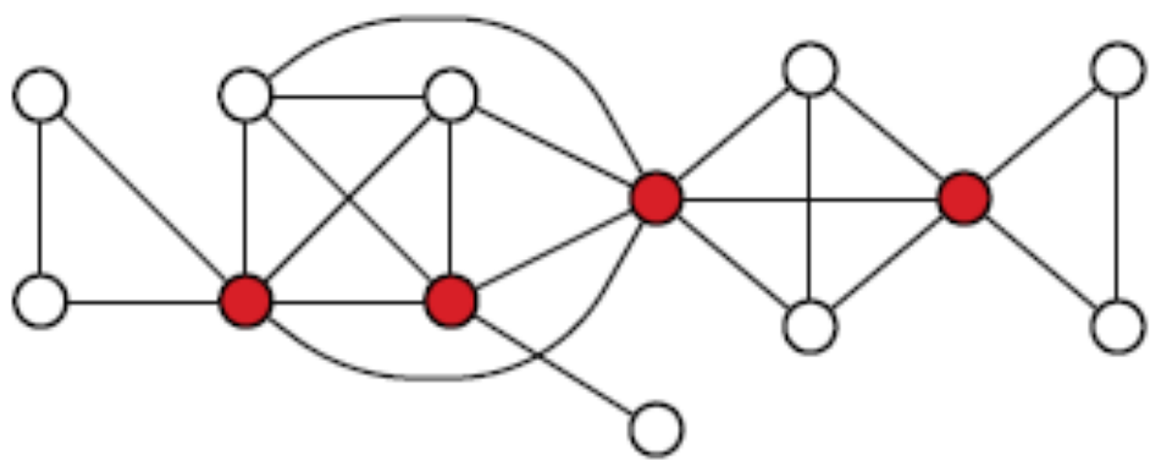
$P_1 = \{\langle u, a, d, v\rangle, \langle u, b, e, v\rangle\}$ $\quad S_1 = \{a, b, c\}$ и $S_2 = \{d, e\}$

$P_2 = \{\langle u, b, d, v\rangle, \langle u, c, e, v\rangle\}$

$\langle u, c, b, d, v\rangle$

Пример разделяющих вершин

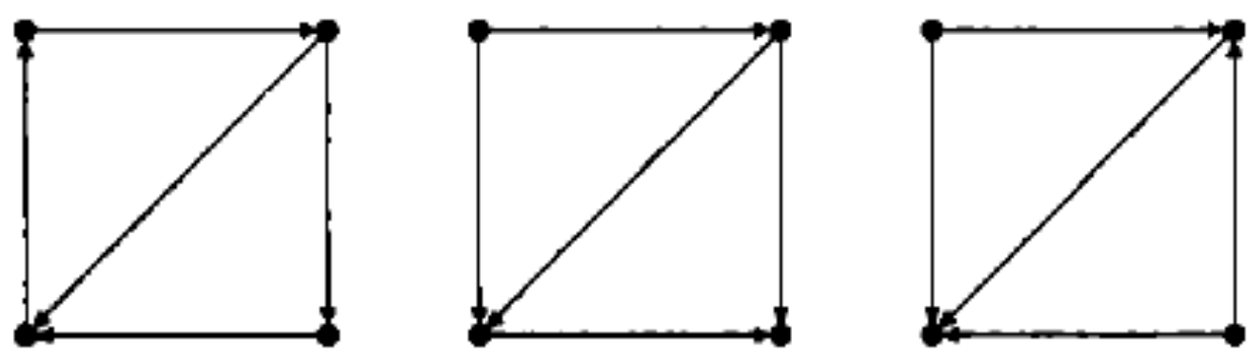




Связность в орграфах

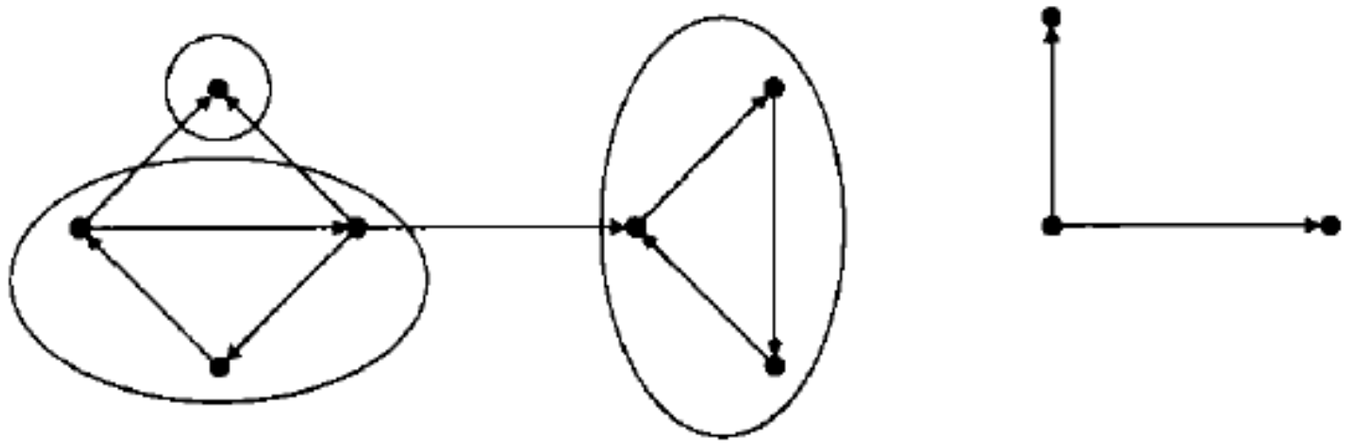
- Связность является одним из немногих понятий, которые не распространяются непосредственно с графов па другие родственные объекты и требуют отдельного определения и рассмотрения в каждом случае. Связность в орграфах чаще всего применяется в программировании.

Пусть $G(V, E)$ — орграф, v_1 и v_2 — его узлы. Говорят, что два узла v_1 и v_2 *сильно связаны* в орграфе G, если существует путь (ориентированная цепь) как из v_1 в v_2, так и из v_2 в v_1. Говорят, что два узла v_1 и v_2 *односторонне связаны* в орграфе G, если существует путь либо из v_1 в v_2, либо из v_2 в v_1. Говорят, что два узла v_1 и v_2 *слабо связаны* в орграфе G, если они связаны в графе G', полученном из G забыванием ориентации дуг. Если все вершины в орграфе сильно (односторонне, слабо) связаны, то орграф называется *сильно* (*односторонне*, *слабо*) *связным*. Сильная связность влечет одностороннюю связность, которая влечет слабую связность. Обратное, разумеется, неверно.

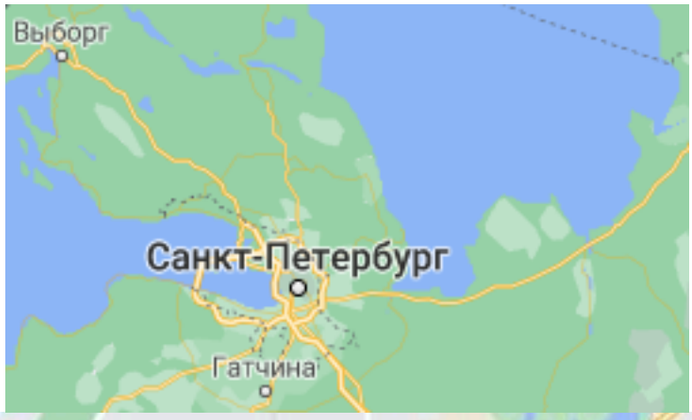


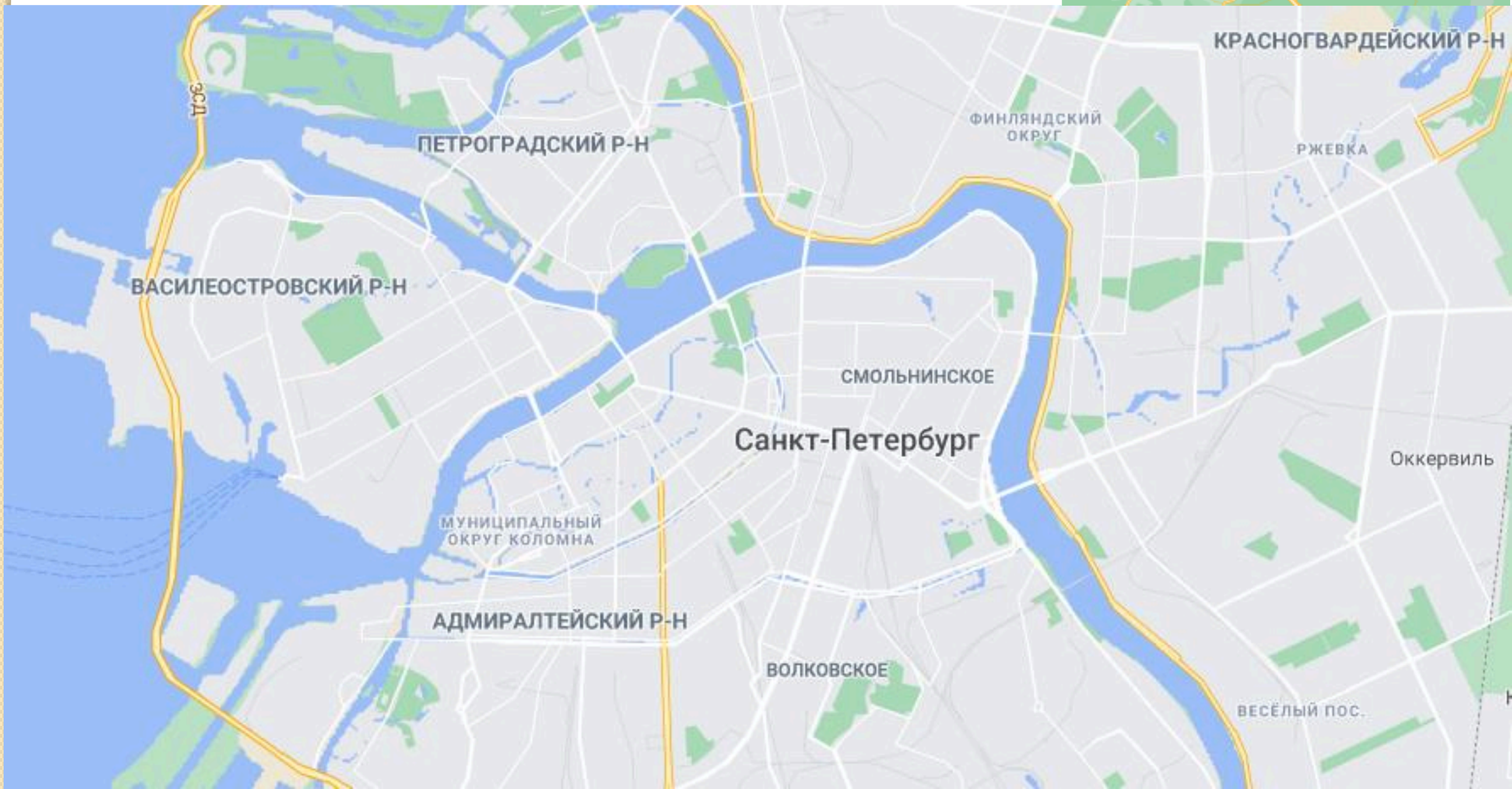
Компоненты сильной связности

- *Компонента сильной связности* (употребляется аббревиатура КСС) орграфа *G* — это его максимальный сильно связный подграф.
- Каждый узел орграфа принадлежит только одной КСС. Если узел не связан сильно с другими, то считаем, что он сам образует КСС. *Конденсацией G** орграфа *G* (или *графом Герца*, или *фактор-графом)* называется орграф, который получается стягиванием в один узел каждой КСС орграфа *G.*
- Фактор-граф не содержит контуров.
- *Вспоминаем* ***фактор-множество***!

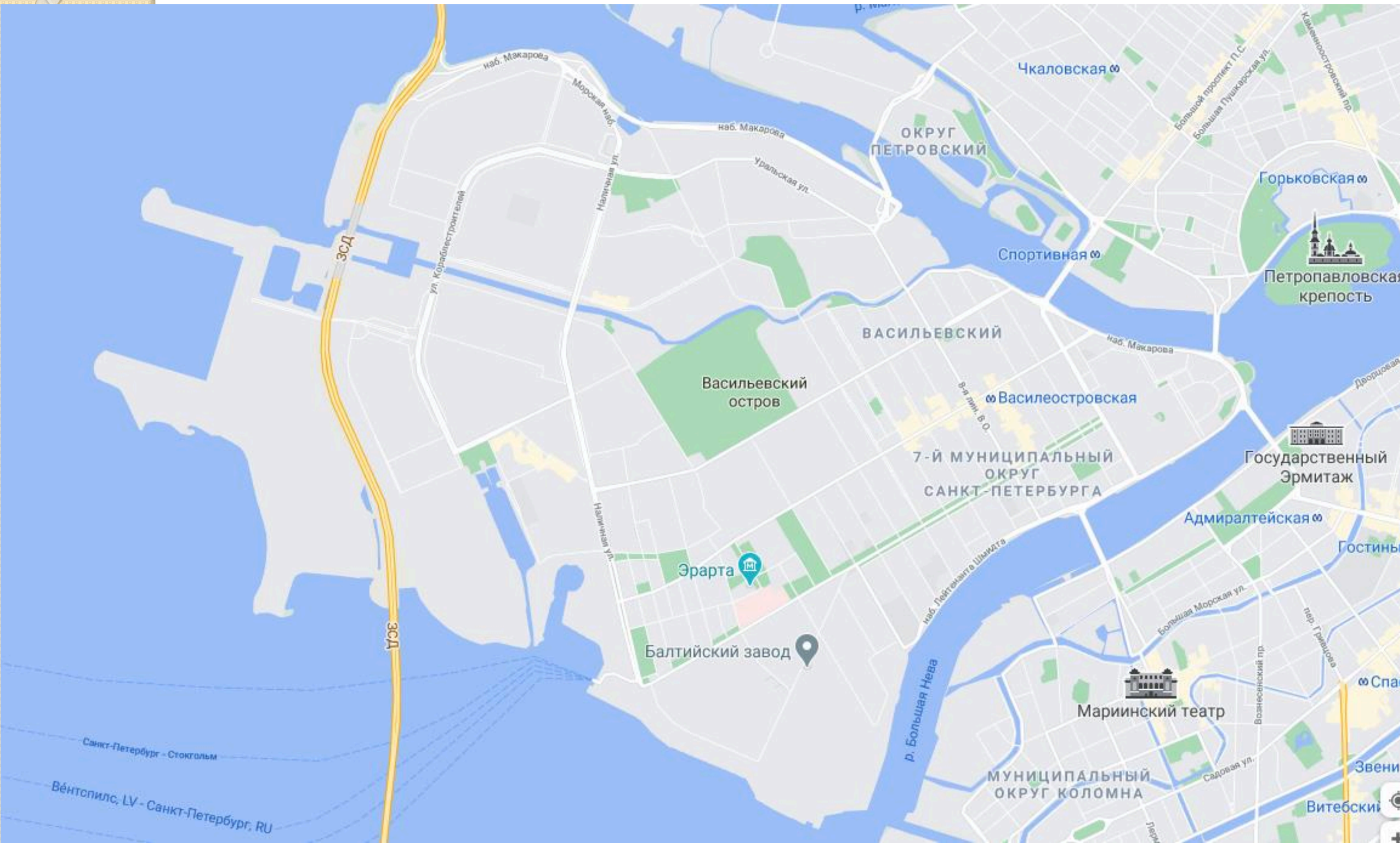


Конденсация графа





Компонента сильной связности?



Выделение компонент сильной связности

Вход: орграф $G(V, E)$, заданный списками смежности $\Gamma(v)$.
Выход: список C компонент сильной связности, каждый элемент которого есть список узлов, входящих в компоненту сильной связности.

```
C := ∅
for v ∈ V do
    M[v] := {v} { M[v] список узлов, входящих в ту же КСС, что и v }
    e[v] := 0 { все узлы не рассмотрены }
end for
while V ≠ ∅ do
    select v ∈ V { взять v из V }
    v → T { положить v в стек }
    e[v] := 1 { отметить v }
    КСС { вызов процедуры КСС }
end while
```

Процедура КСС

```
if T = ∅ then
   return { негде выделять }
end if
v := top T { v — верхний элемент стека}
if Γ[v] ∩ V = ∅ then
   C := C ∪ M[v] { это КСС }
   V := V − v { удалить узел }
   v ← T { снять узел v со стека }
   КСС { возврат из тупика }
else
   for u ∈ Γ[v] do
      if e[u] = 0 then
         u → T { положить узел u в стек }
         e[u] := 1 { отметить узел u }
      else
         repeat
            w ← T { w — склеиваемый узел }
            V := V − w { удалить узел }
            Γ[u] := Γ[u] ∪ Γ[w] { запомнить смежность }
            M[u] := M[u] ∪ M[w] { склеивание узлов }
         until u = w
         w → T { чтобы не убрать тот узел, }
         V := V + w { с которого начали }
      end if
      КСС { поиск в глубину }
   end for
end if
```

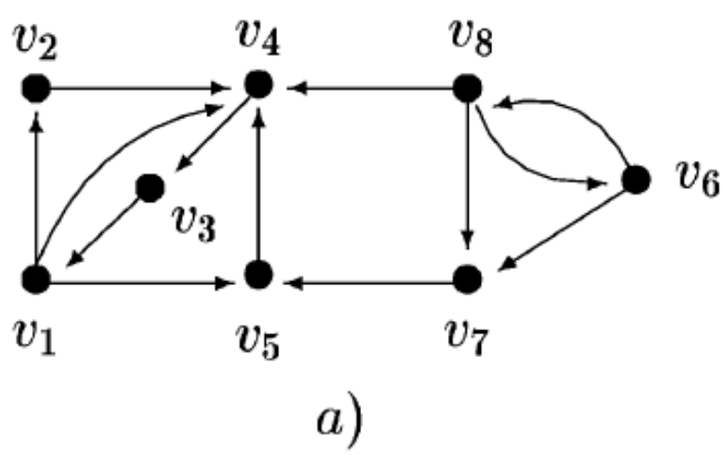
Достаточно заметить, что любой контур принадлежит ровно одной КСС, и более того, если в КСС входит несколько узлов, то они обязательно принадлежат некоторым контурам. Поэтому, если при обходе в глубину мы попадаем в уже отмеченный узел ***u,*** *то это означает, что обнаружен контур, причём предшествующие узлы найденного контура находятся на стеке, начиная от вершины стека и до рассматриваемого отмеченного узла* ***u,*** *который также присутствует в стеке. Все узлы контура найденного контура можно «склеить».*

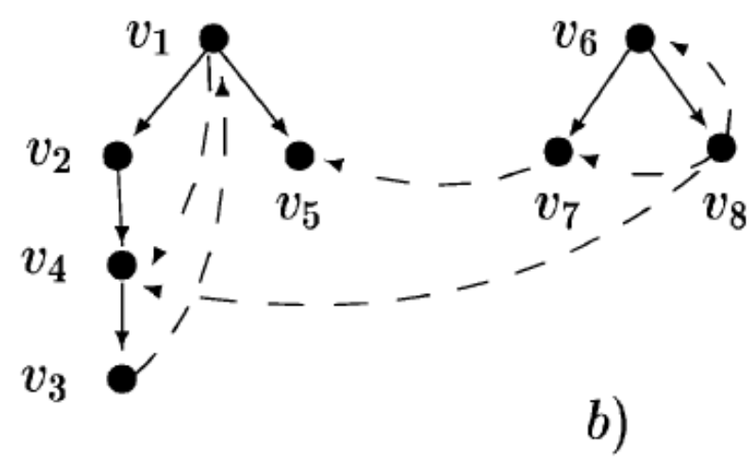
Выделение компонент сильной связности в орграфе

Алгоритм поиска в глубину строит *глубинный лес*, состоящий из ориентированных корневых деревьев.

Разбивает на 4 класса:

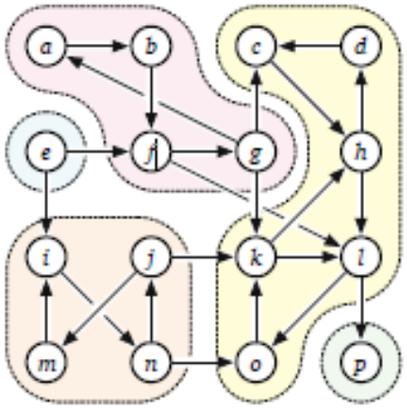
- древесные дуги, идущие от предка к потомку (отец и сын);
- обратные дуги, от потомка к предку;
- прямые дуги, идущие от предка к потомку, но не являющие древесными;
- поперечные дуги, соединяющие вершины, ни одна из которых не является потомком другой.

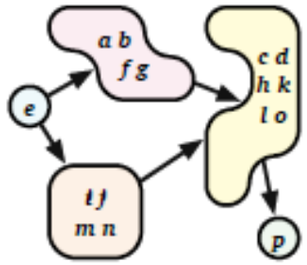




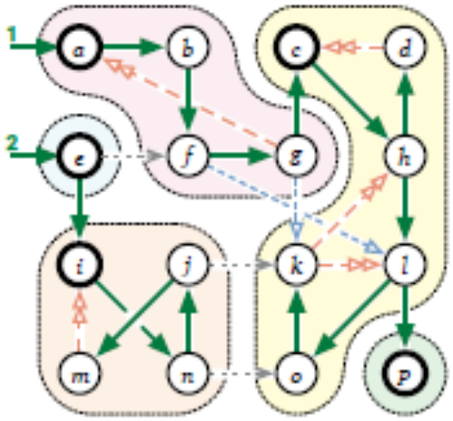
Пример КСС, или Сильные компоненты графа

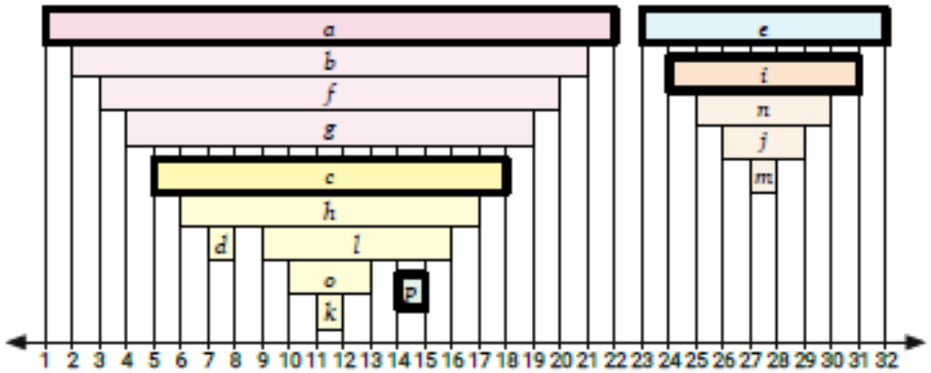
- Лемма. Пусть определен обход в глубину в любом направлении в графе G. Каждая сильная компонента содержит ровно один узел, который не имеет предка в C (этот узел либо имеет предка в другой сильной компоненте, либо не имеет предка в принципе.



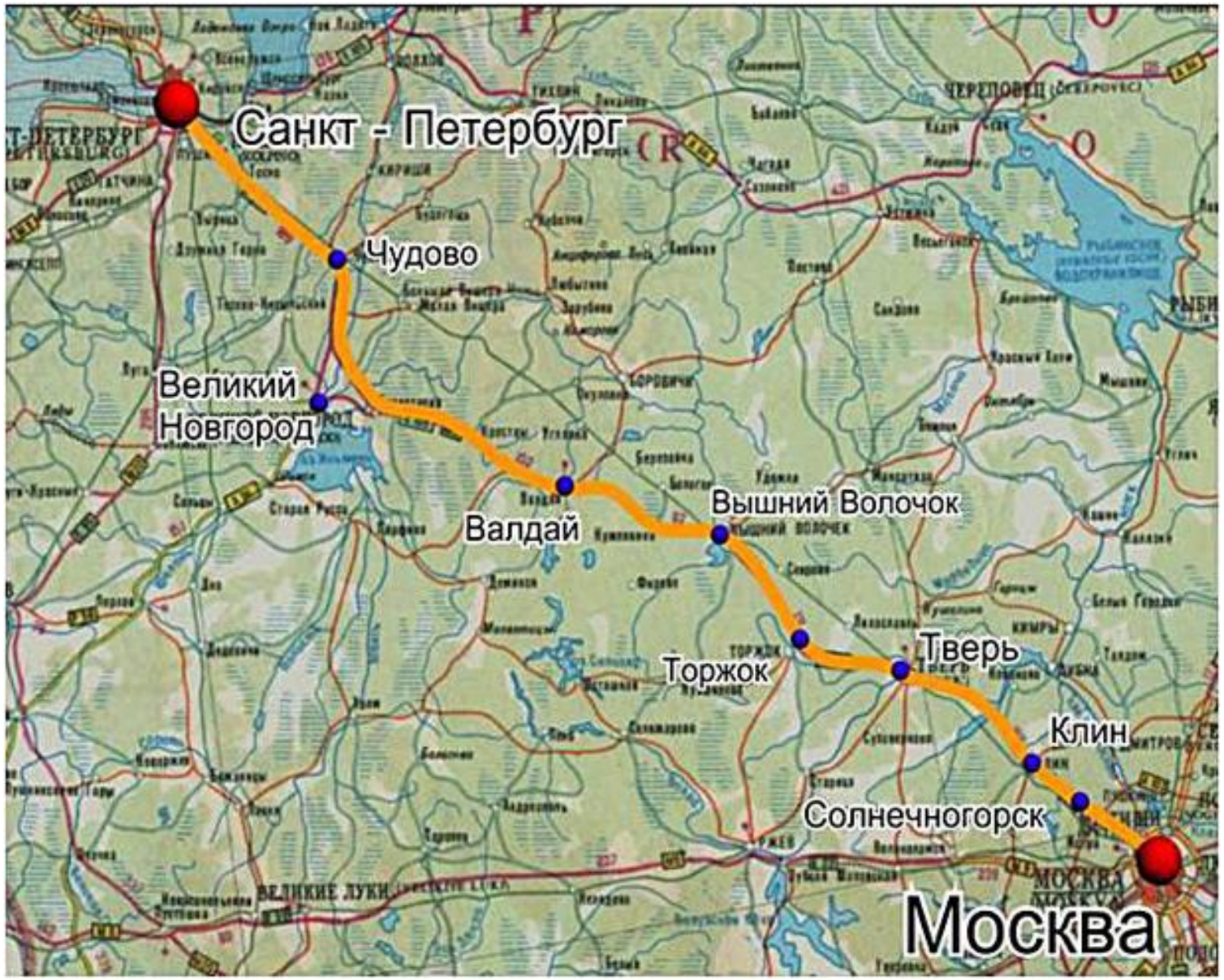


Сильные компоненты непрерывны в лесу поиска в глубину.






```
STRONGCOMPONENTS(G):
  count ← 0
  while G is non-empty
      C ← ∅
      count ← count + 1
      v ← any vertex in a sink component of G     ⟨⟨Magic!⟩⟩
      for all vertices w in reach(v)
          w.label ← count
          add w to C
      remove C and its incoming edges from G
```


Санкт - Петербург
Чудово
Великий Новгород
Валдай
Вышний Волочок
Торжок
Тверь
Клин
Солнечногорск
Москва

Теорема Менгера

- Т Е О Р Е М А (Менгера). *Пусть **u** и **v** — несмежные вершины в графе G. Наименьшее число вершин в множестве, разделяющем **u** и **v**, равно наибольшему числу вершинно непересекающихся простых **<u, v>**-цепей:*

$$\max |P(u,v)| = \min |S(u,v)|.$$

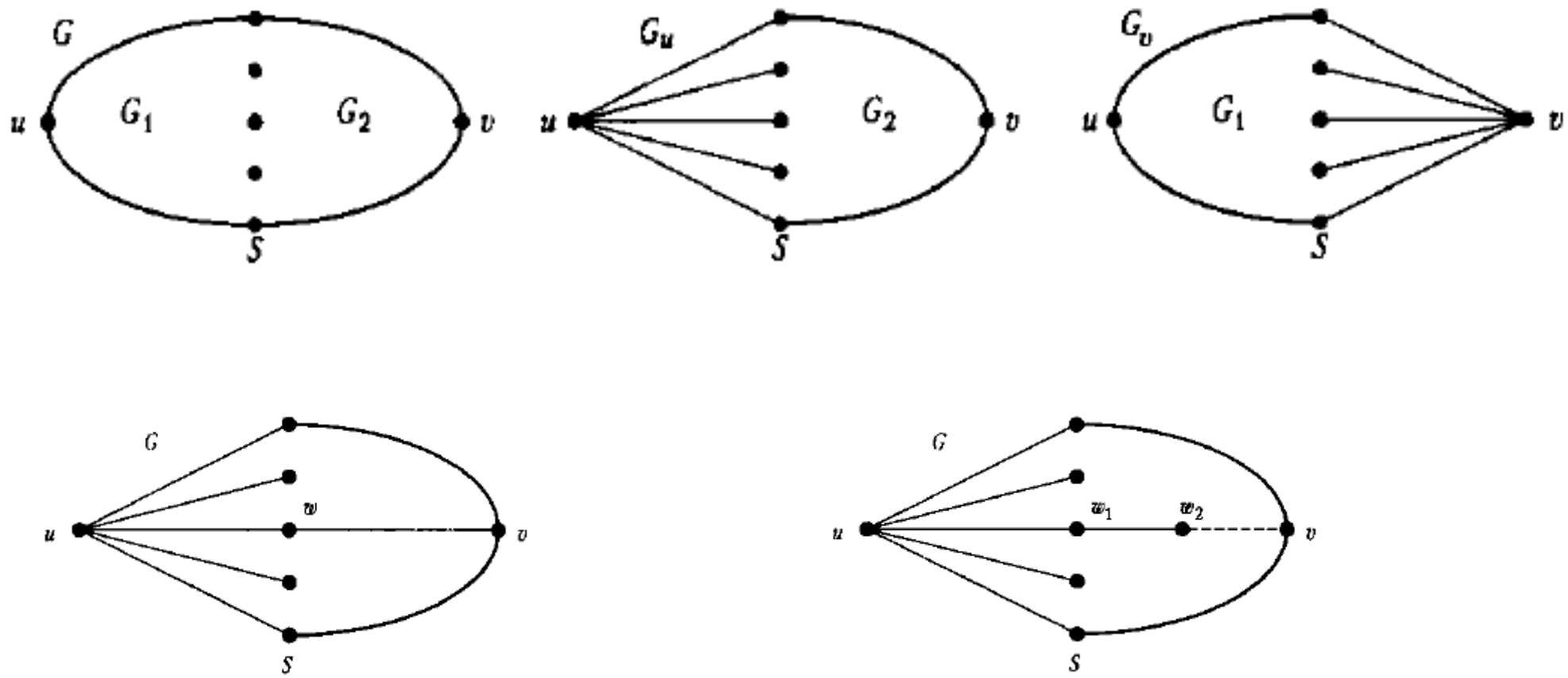
$$\forall P \ (\forall S \ (|P| \leqslant |S|)).$$

- ТЕОРЕМА *Для любых двух несмежных вершин **u** и **v** графа **G** наибольшее число рёберно-непересекающихся **<u, v>** -цепей равно наименьшему числу рёбер в **<u, v>** -разрезе.*

- ТЕОРЕМА *Чтобы граф **G** был **n**-связным, необходимо и достаточно, чтобы любые две несмежные вершины были соединены не менее чем **n** вершинно-непересекающимися простыми цепями.*

В любом графе ***G*** любые две несмежные вершины соединены не менее чем ***χ(G)*** вершинно-непересекающимися простыми цепями.

Пояснение к доказательству



Теорема Холла

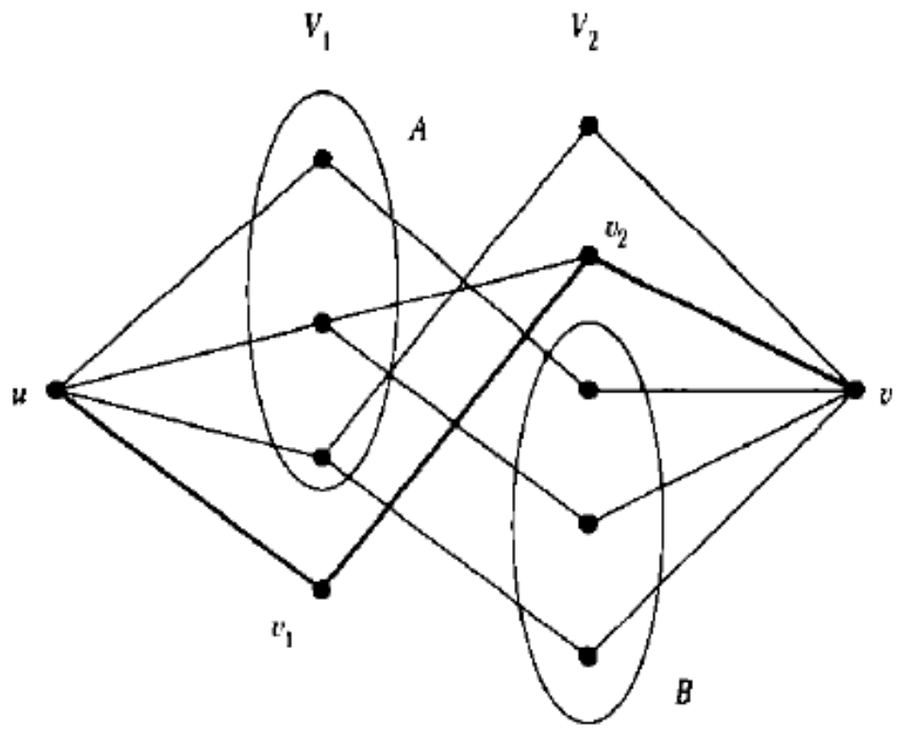
Задача о свадьбах

- Пусть имеется конечное множество юношей, каждый из которых знаком с некоторым подмножеством конечного множества девушек. В каком случае всех юношей можно женить так, чтобы каждый женился на знакомой девушке?
- *Паросочетанием* (или *независимым множеством рёбер*) называется множество рёбер, в котором никакие два ребра не смежны. Паросочетание называется *максимальным*, если никакое его надмножество не является независимым.
- Пусть *G(V*1, V2, *E)* — двудольный граф. *Совершенным паросочетанием* из V1 в *V2* называется паросочетание, покрывающее вершины V1. В каком случае существует совершенное паросочетание из V1 в *V2?*
- **Совершенное паросочетание является максимальным.**

- *Пусть G(V1, V2, E) — двудольный граф. Совершенное паросочетание из* V1 *в V2 существует тогда и только тогда, когда*

$$\forall A \subset V_1 \ \ (|A| \leqslant |\Gamma(A)|).$$

Теорема Холла



Генератор случайных графов

$$[0,\ n]$$

$$\lfloor nR \rfloor + 1$$

$$N = (n-1)i + j$$

$$N = \lfloor n^2 R \rfloor + 1$$

$$i = \left\lfloor \frac{N}{n} \right\rfloor + 1, \quad j = N - (i-1)n.$$

Алгоритм построения случайного неориентированного графа

```
Input
    n — количество вершин графа;
    m — количество ребер графа;
    M — матрица размера n × n,
        содержащая во всех ячейках Л;
    begin
        for k = 1 to m do
            begin
                получить с помощью датчика
                случайное число R;
                N := ⌊n²R⌋ + 1;
                i := ⌊N : n⌋ + 1;
                j := N − (i − 1) · n;
                if M(i, j) ≠ И and i ≠ j then
                else
                    m := m + 1
            end
    end
Output M — матрица смежности неориентированного графа.
```

Алгоритм построения случайного ориентированного графа

```
Input
    n — количество вершин орграфа;
    m — количество дуг орграфа;
    M — матрица размера n × n,
        содержащая во всех ячейках Л;
    begin
        for k = 1 to m do
            begin
                получить с помощью датчика
                случайное число R;
                N := ⌊n²R⌋ + 1;
                i := ⌊N : n⌋ + 1;
                j := N − (i − 1) · n;
                if M(i, j) ≠ И and i ≠ j then
                    begin
                     M(i, j) := И;
                    end
                else
                     m := m + 1
            end
    end
Output M — матрица смежности орграфа.
```

Алгоритм построения случайного ориентированного бесконтурного графа

```
Input
    n — количество вершин орграфа;
    m — количество дуг орграфа;
    M — матрица размера n × n,
        содержащая во всех ячейках Л;
    begin
        for k = 1 to m do
            begin
                получить с помощью датчика
                случайное число R;
                N := ⌊n^2 R⌋ + 1;
                i := ⌊N : n⌋ + 1;
                j := N − (i − 1) · n;
                if i ≠ j and i < j and M(i, j) ≠ И then
                 M(i, j) := И;
                if i ≠ j and i > j and M(i, j) ≠ И then
                 M(j, i) := И;
                else
                 m := m + 1
            end
    end
Output M — матрица смежности бесконтурного орграфа.
```

Пример работы алгоритма (n=5, m=8)

$$N = \lfloor n^2 R \rfloor + 1$$

$$i = \left\lfloor \frac{N}{n} \right\rfloor + 1, \quad j = N - (i-1)n.$$

k	R	N	i	j	m	M
1	0,21132	6	2	1	8	$\begin{bmatrix} Л & Л & Л & Л & Л \\ И & Л & Л & Л & Л \\ Л & Л & Л & Л & Л \\ Л & Л & Л & Л & Л \\ Л & Л & Л & Л & Л \end{bmatrix}$
2	0,26215	7	2	2	9	$\begin{bmatrix} Л & Л & Л & Л & Л \\ И & Л & Л & Л & Л \\ Л & Л & Л & Л & Л \\ Л & Л & Л & Л & Л \\ Л & Л & Л & Л & Л \end{bmatrix}$
3	0,79253	20	4	5	9	$\begin{bmatrix} Л & Л & Л & Л & Л \\ И & Л & Л & Л & Л \\ Л & Л & Л & Л & Л \\ Л & Л & Л & Л & И \\ Л & Л & Л & Л & Л \end{bmatrix}$
4	0,28052	8	2	3	9	$\begin{bmatrix} Л & Л & Л & Л & Л \\ И & Л & И & Л & Л \\ Л & Л & Л & Л & Л \\ Л & Л & Л & Л & И \\ Л & Л & Л & Л & Л \end{bmatrix}$
5	0,93648	24	5	4	10	$\begin{bmatrix} Л & Л & Л & Л & Л \\ И & Л & И & Л & Л \\ Л & Л & Л & Л & Л \\ Л & Л & Л & Л & И \\ Л & Л & Л & И & Л \end{bmatrix}$

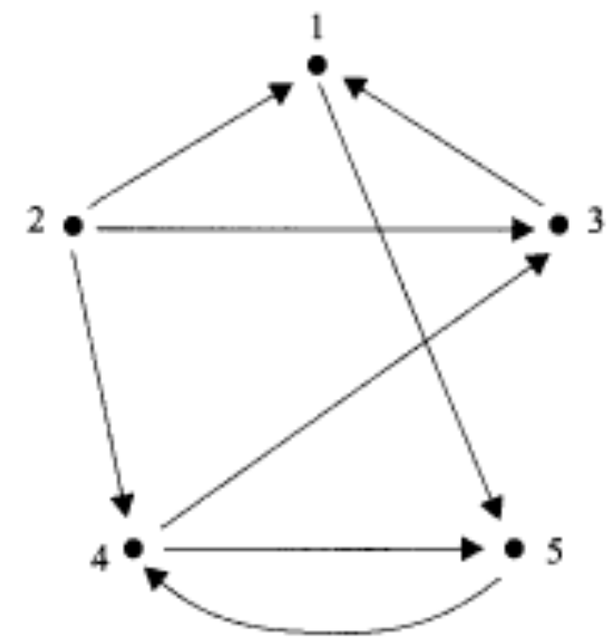
Пример работы алгоритма (n=5, m=8)

$$N = \lfloor n^2 R \rfloor + 1$$

$$i = \left\lfloor \frac{N}{n} \right\rfloor + 1, \quad j = N - (i-1)n.$$

7	0,35606	9	2	4	10	$\begin{bmatrix} Л & Л & Л & Л & Л \\ И & Л & И & И & Л \\ Л & Л & Л & Л & Л \\ Л & Л & Л & Л & И \\ Л & Л & Л & И & Л \end{bmatrix}$
8	0,16043	5	1	5	10	$\begin{bmatrix} Л & Л & Л & Л & И \\ И & Л & И & И & Л \\ Л & Л & Л & Л & Л \\ Л & Л & Л & Л & И \\ Л & Л & Л & И & Л \end{bmatrix}$
9	0,40480	11	3	1	10	$\begin{bmatrix} Л & Л & Л & Л & И \\ И & Л & И & И & Л \\ И & Л & Л & Л & Л \\ Л & Л & Л & Л & И \\ Л & Л & Л & И & Л \end{bmatrix}$
10	0,74225	19	4	4	11	$\begin{bmatrix} Л & Л & Л & Л & И \\ И & Л & И & И & Л \\ И & Л & Л & Л & Л \\ Л & Л & Л & Л & И \\ Л & Л & Л & И & Л \end{bmatrix}$
11	0,70183	18	4	3	11	$\begin{bmatrix} Л & Л & Л & Л & И \\ И & Л & И & И & Л \\ И & Л & Л & Л & Л \\ Л & Л & И & Л & И \\ Л & Л & Л & И & Л \end{bmatrix}$

Пример работы алгоритма (n=5, m=8)



$$N = \lfloor n^2 R \rfloor + 1$$

$$i = \left\lfloor \frac{N}{n} \right\rfloor + 1, \quad j = N - (i-1)n.$$

Применение метода Шимбелла

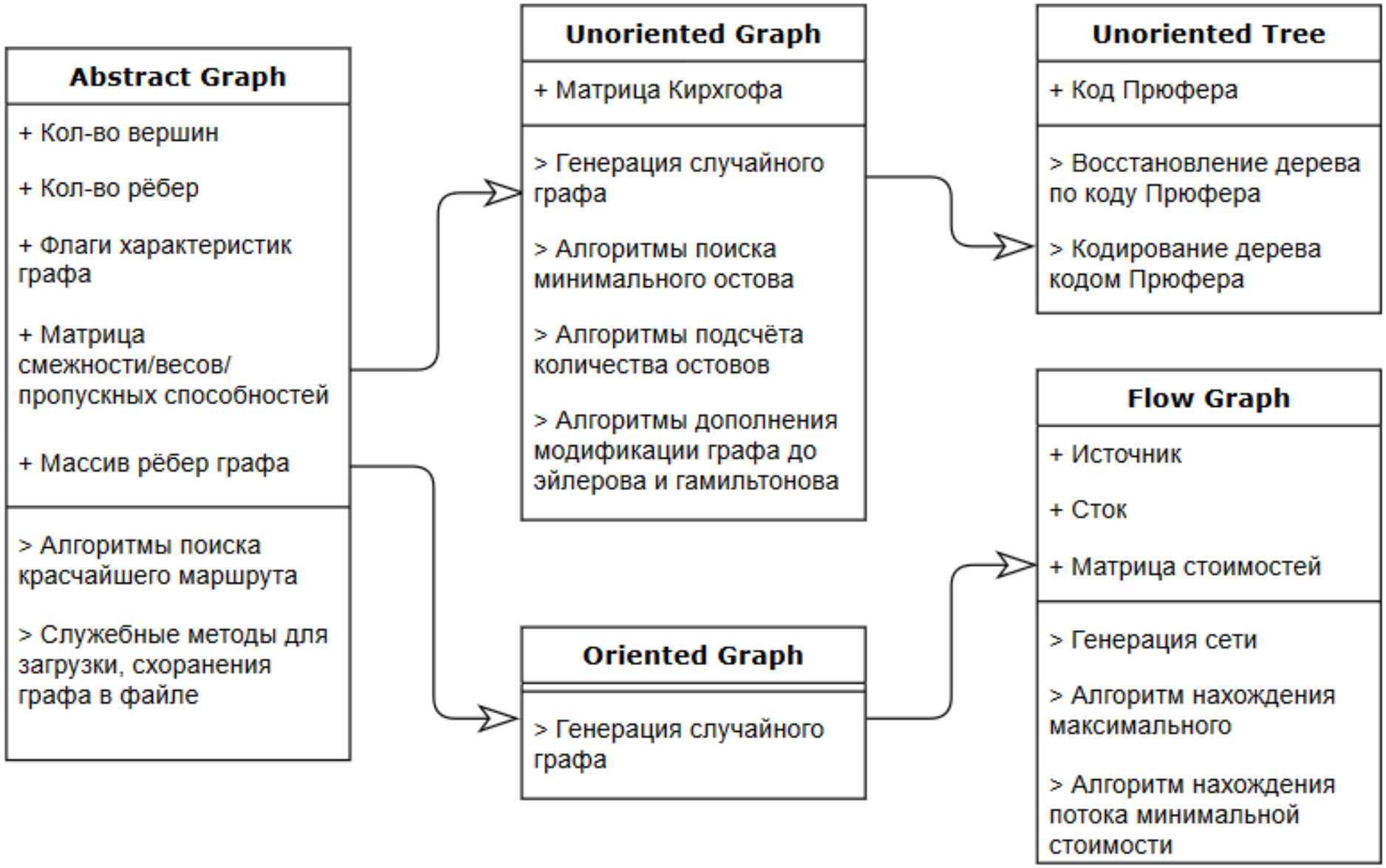
- *Теорема*. Для определения количества маршрутов, состоящих из ***к*** ребер (дуг), необходимо возвести в ***к-ую*** степень матрицу смежности вершин. Тогда соответствующий элемент даст количество маршрутов длины ***к*** (состоящих из ***к*** ребер).

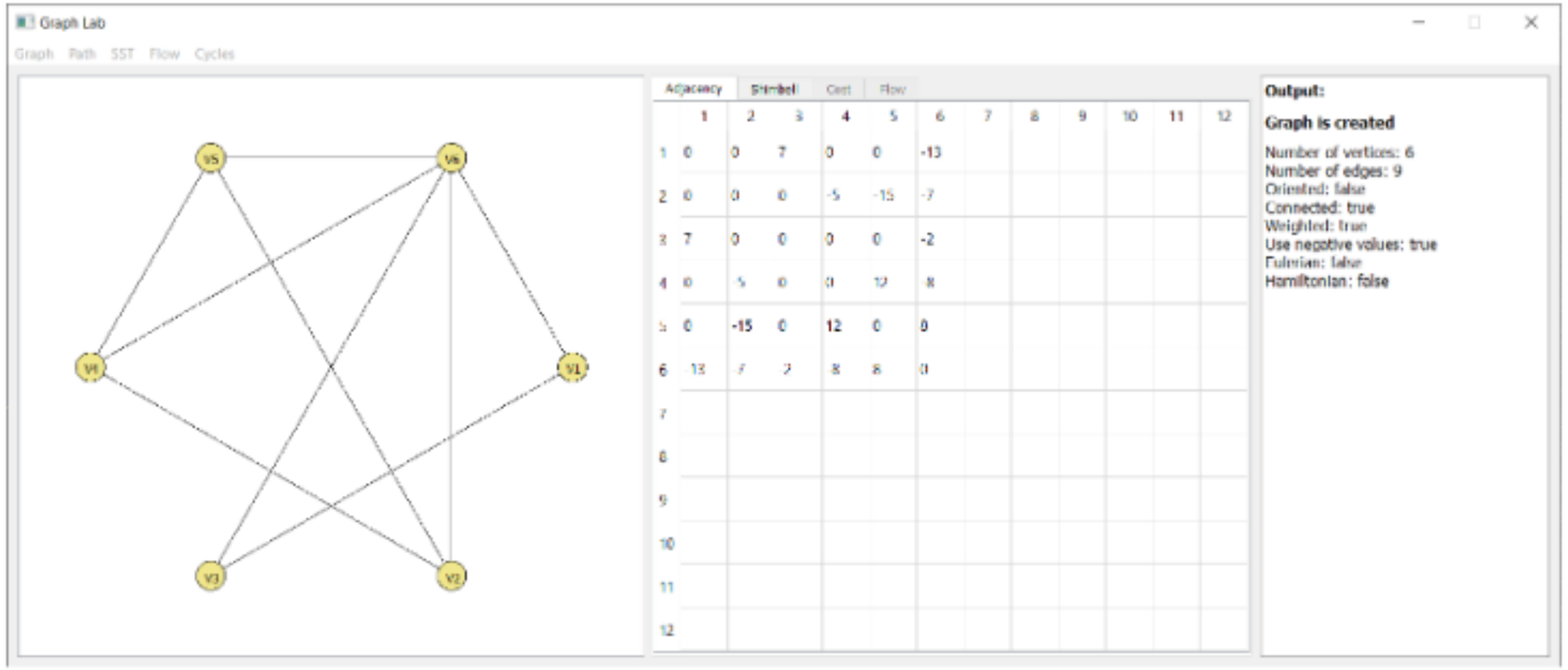
Следствие 1. **В графе** G **мощности** n **тогда и только тогда существует** (x_i, x_j)**-маршрут** $(x_i \neq x_j)$**, когда** (i, j)**-й элемент матрицы** $P + P^2 + \ldots + P^{n-1}$ **не равен нулю.**

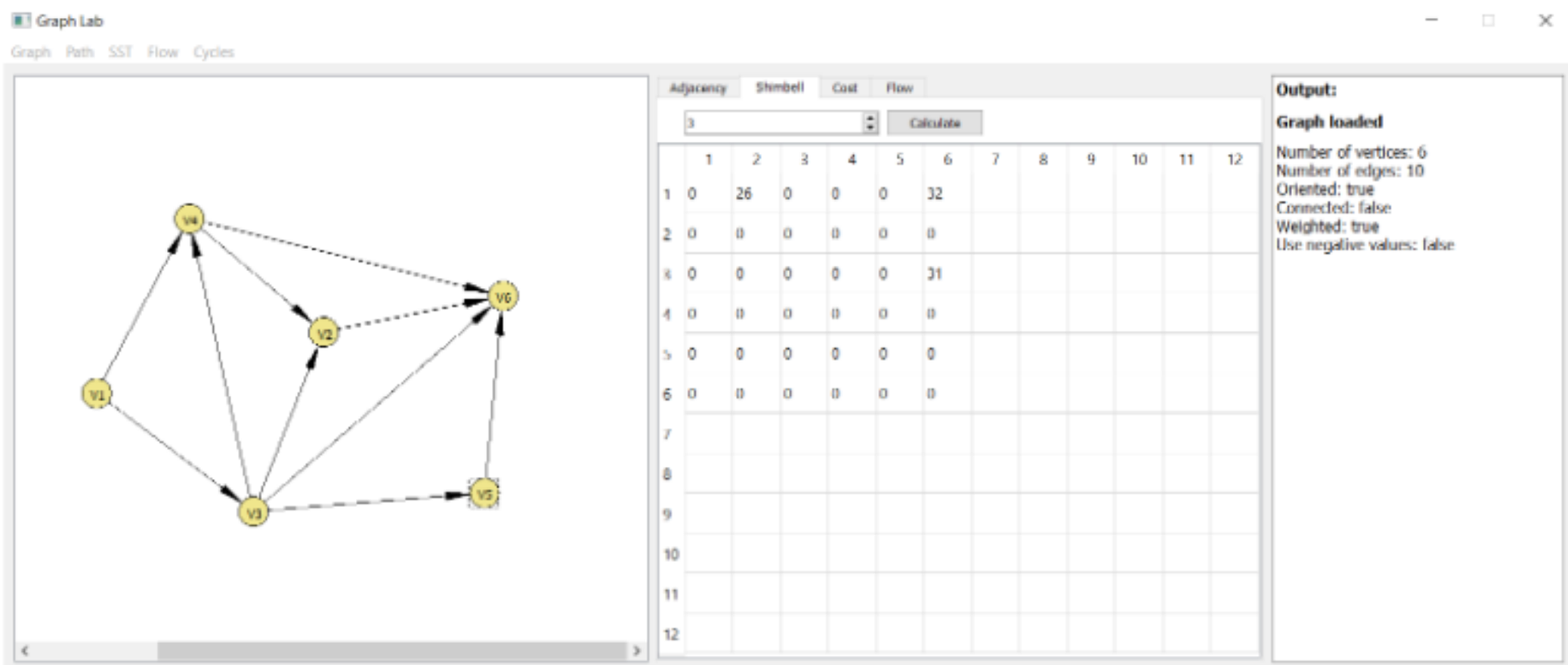
Следствие 2. **В графе** G **мощности** n **тогда и только тогда существует цикл, содержащий вершину** x_i**, когда** (i, i)**-й элемент матрицы** $P + P^2 + \ldots + P^n$ **не равен нулю.**

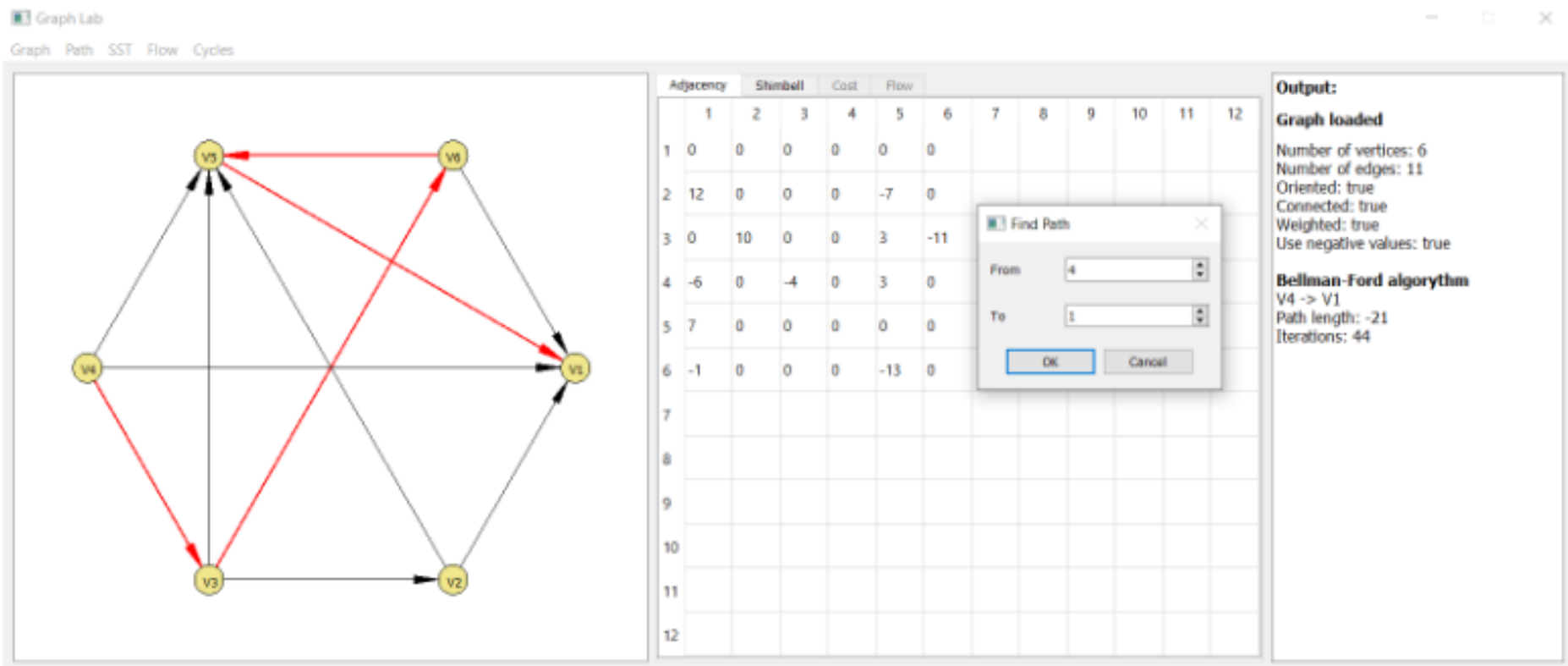
Для ацикличности графа – треугольная матрица смежности вершин.
Для связности графа – сложение всех степеней матрицы вершин от 1 до p-1.

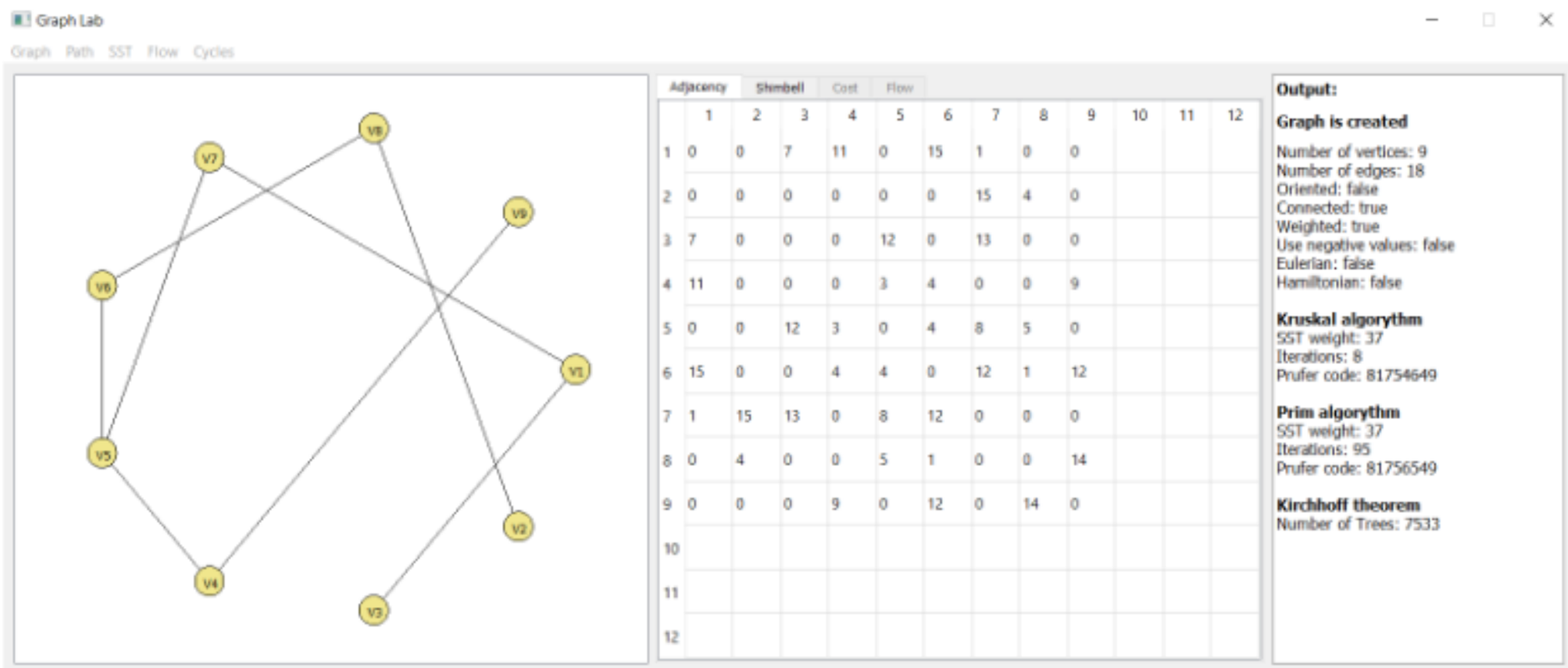
Иерархия и структура классов




Graph Lab
Graph Path SST Flow Cycles
Adjacency Shimbell Cost Flow
Output:
Graph is created
Number of vertices: 6
Number of edges: 9
Oriented: false
Connected: true
Weighted: true
Use negative values: true
Eulerian: false
Hamiltonian: false


Graph Lab
Graph Path SST Flow Cycles
Adjacency Shimbell Cost Flow
Calculate
Output:
Graph loaded
Number of vertices: 6
Number of edges: 10
Oriented: true
Connected: false
Weighted: true
Use negative values: false


Graph Lab
Graph Path SST Flow Cycles
V5
V6
V4
V1
V3
V2
Adjacency Shimbell Cost Flow
1 2 3 4 5 6 7 8 9 10 11 12
1 0 0 0 0 0 0
2 12 0 0 0 -7 0
3 0 10 0 0 3 -11
4 -6 0 -4 0 3 0
5 7 0 0 0 0 0
6 -1 0 0 0 -13 0
7
8
9
10
11
12
Find Path
From 4
To 1
OK
Cancel
Output:
Graph loaded
Number of vertices: 6
Number of edges: 11
Oriented: true
Connected: true
Weighted: true
Use negative values: true
Bellman-Ford algorythm
V4 -> V1
Path length: -21
Iterations: 44


Graph Lab
Graph Path SST Flow Cycles
V7
V8
V9
V6
V1
V5
V2
V4
V3
Adjacency Shimbell Cost Flow
1 2 3 4 5 6 7 8 9 10 11 12
1 0 0 7 11 0 15 1 0 0
2 0 0 0 0 0 0 15 4 0
3 7 0 0 0 12 0 13 0 0
4 11 0 0 0 3 4 0 0 9
5 0 0 12 3 0 4 8 5 0
6 15 0 0 4 4 0 12 1 12
7 1 15 13 0 8 12 0 0 0
8 0 4 0 0 5 1 0 0 14
9 0 0 0 9 0 12 0 14 0
10
11
12
Output:
Graph is created
Number of vertices: 9
Number of edges: 18
Oriented: false
Connected: true
Weighted: true
Use negative values: false
Eulerian: false
Hamiltonian: false
Kruskal algorythm
SST weight: 37
Iterations: 8
Prufer code: 81754649
Prim algorythm
SST weight: 37
Iterations: 95
Prufer code: 81756549
Kirchhoff theorem
Number of Trees: 7533

СПАСИБО
ЗА ВНИМАНИЕ